📄 WEEKLY PROGRESS REPORT – MINOR PROJECT

Project Title

- AI-Powered Student Assistance Chatbot for Department of Technical Education, Government of Rajasthan

Group ID

- G-1

Branch & Section

- AI & DS – B

Date Range

- 23/02/2026 to 28/02/2026

PHASE 3: OBJECT-ORIENTED DESIGN
Week 6: Detailed Class Design & Packaging

1️⃣ EXECUTIVE SUMMARY

Current Sprint

- Sprint 6

Sprint Goal

- Detailed Class Design & System Packaging

Status

- On Track

Summary of Work Done

During Week 6, the team focused on refining the structural design of the system by creating a Detailed Class Diagram including attributes, methods, visibility modifiers, and interfaces.

SOLID principles were applied to ensure the system design remains modular, scalable, and maintainable.

A Package Diagram was also created to organize related classes into logical modules such as UI, Business Logic, Data Access, and Admin modules.

2️⃣ UML DELIVERABLES

1. Detailed Class Diagram

The updated Class Diagram now includes:

🔹 Classes Identified:

1️⃣ User
- userId : int
- name : String
- role : String
- sendQuery()

receiveResponse()

2️⃣ ChatbotController
- nlpProcessor : NLPProcessor
- queryService : QueryService
- processQuery(query : String) : String
- validateQuery() : boolean

3️⃣ NLPProcessor
- modelType : String
- extractIntent(query : String) : String
- extractEntities(query : String) : List

4️⃣ QueryService
- databaseService : DatabaseService
- fetchCollegeInfo()
- fetchAdmissionDetails()
- fetchFees()
- fetchScholarship()
- fetchPlacement()

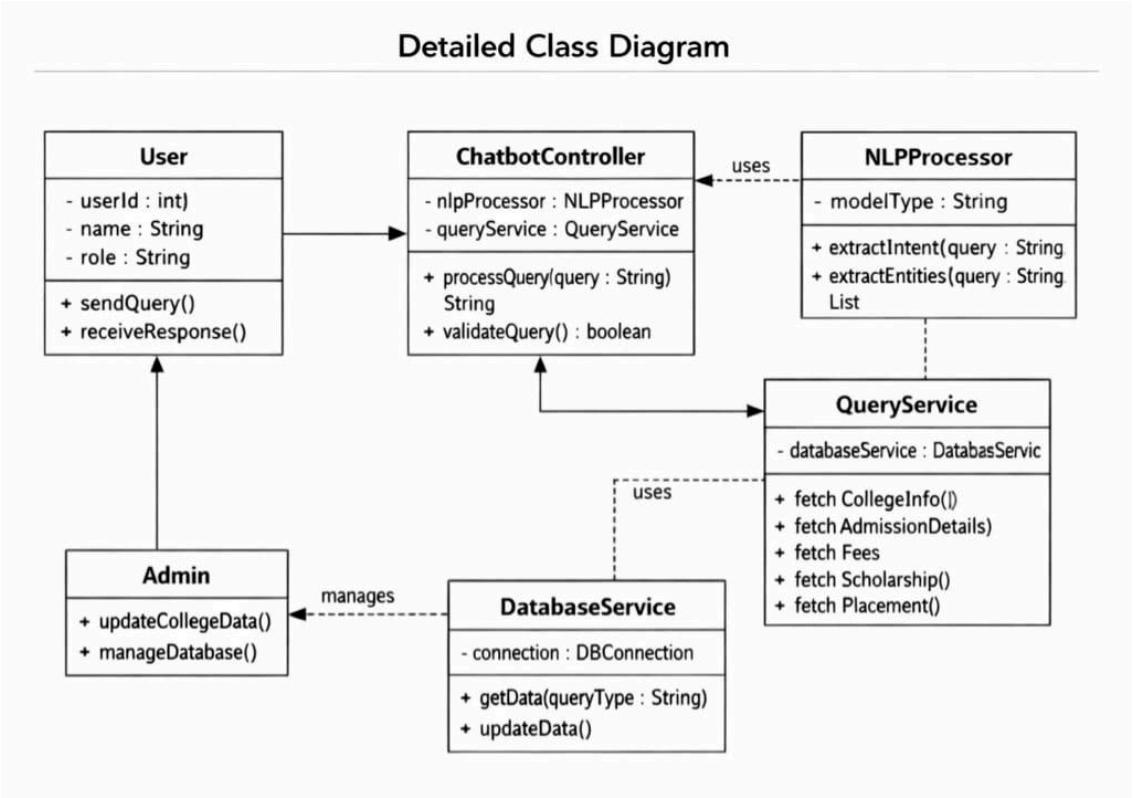
5️⃣ DatabaseService
- connection : DBConnection
- getData(queryType : String)
- updateData()

6️⃣ Admin
- updateCollegeData()
- manageDatabase()

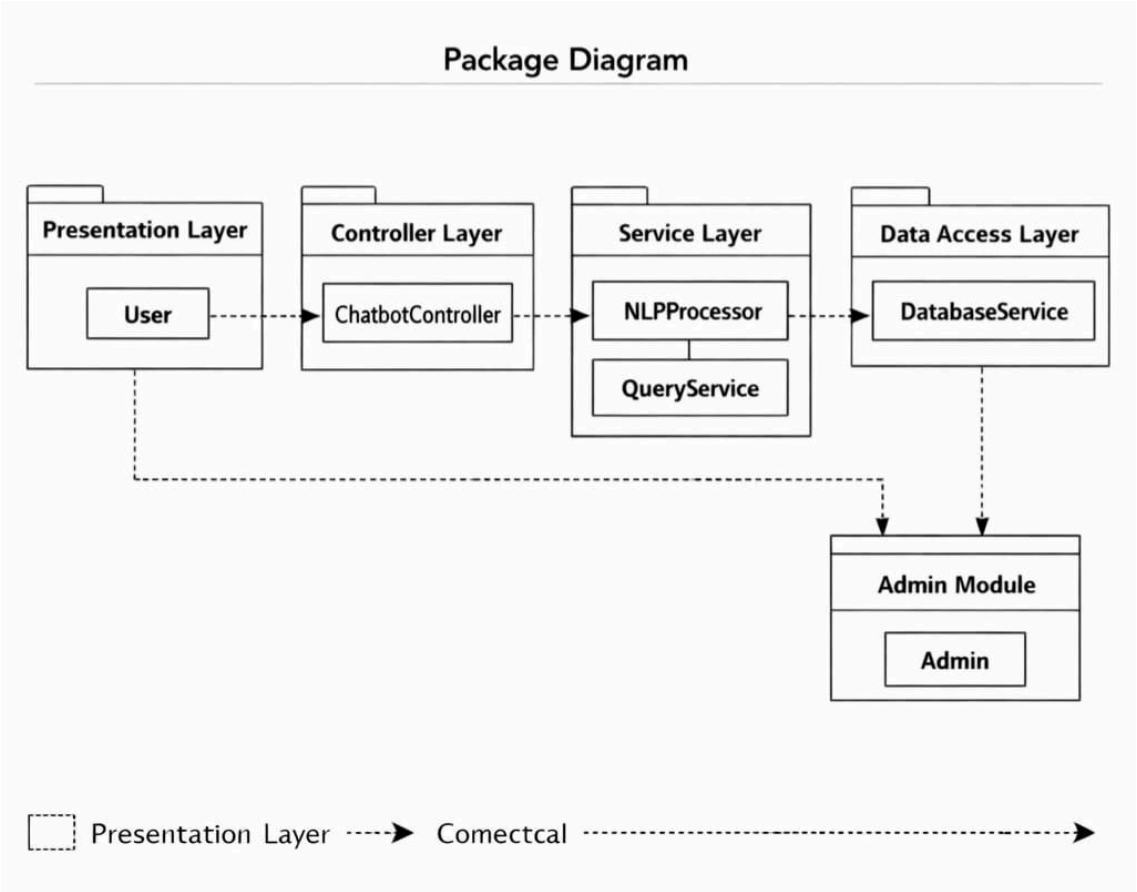
🔹 Visibility Used:

+ Public
- Private
Protected

- **Detaiiled class diagram**



- **Package diagram**



✅ SOLID Principles Applied

1. Single Responsibility Principle
Each class has one clear responsibility (e.g., NLP only processes language).

2. Open/Closed Principle
System designed to allow new query types without modifying core logic.

3. Liskov Substitution Principle
Admin and User follow consistent interaction model.

4. Interface Segregation Principle
Separate service interfaces for query handling and data management.

5. Dependency Inversion Principle
High-level modules depend on abstractions (e.g., QueryService uses Database interface).

1. Design Patterns Used

MVC Pattern (UI – Controller – Model separation)

Factory Pattern (for creating response objects)

Service Layer Pattern (QueryService layer)

2. Package Diagram

Classes organized into logical packages:

📦 Presentation Layer
- User
- Chat Interface

📦 Controller Layer
- ChatbotController

📦 Service Layer
- NLPProcessor
- QueryService

📦 Data Access Layer
- DatabaseService

📦 Admin Module
- Admin

This improves modularity and maintainability.

3️⃣ TECHNICAL EXECUTION

Jira Updates

- Detailed class design tasks completed

- SOLID review session conducted

Git Updates

- Detailed UML diagrams committed
- Package structure updated

Branch Used

- detailed-design

4 CHALLENGES FACED

- Ensuring low coupling between modules
- Designing scalable service layer
- Structuring class responsibilities clearly

5 GOALS FOR NEXT WEEK (WEEK 7)

- Final architecture validation
- Begin backend implementation
- API development
- Database integration

6 SIGNATURES

Project Guide Signature: _____

Mentor Signature: _____